

A Visualisation Layer for Web-Based Digital Twin Platforms

Problem, Solution Architecture, Techniques, Open-Source Packages and an Implementation Plan for the DTaaS Client

Prepared for the INTO-CPS Association
Digital Twin as a Service (DTaaS) project

13 September 2026

Abstract

The DTaaS web client has no spatial visualisation capability: it is a React 19 / Vite application whose dependency set contains no 3D, image-overlay or scientific-visualisation library. Any digital twin (DT) with a spatial subject can be executed, stored and logged on the platform, but seen only as a time series. This report states the problem (§2), proposes a layered architecture in which the *binding layer* — not the renderer — is the reusable asset (§3), catalogues fourteen visualisation techniques mapped onto the domains DTaaS serves (§4), names an open-source package for each with version and licence (§5), and sets out a phased plan for a React component package, `@dtaas/visualisation` (§6).

Three claims organise it. First, such a package must not be a renderer: renderers are domain-specific and short-lived, whereas anchor resolution, the temporal state store and the encoding vocabulary are durable. Second, a 3D scene is one *substrate* among several — photographs and live video are equally legitimate places to put simulation output, and are what many users ask for first. Third, roughly a third of what such a layer normally builds is already deployed: `dtaas-services` provides RabbitMQ (AMQP and MQTT from one broker), InfluxDB, ThingsBoard, Grafana, MongoDB and GitLab, and the client already ships `@gitbeaker/rest` and `react-iframe`. The plan consumes those rather than duplicating them, and makes visualisation an additional DTaaS library asset type so a configuration can be authored, versioned in Git, reviewed as a merge request, shared and reused exactly as any other DT asset is.

1 Introduction

Digital Twin as a Service (DTaaS) is a platform for the creation, execution and sharing of digital twins, owned by the INTO-CPS Association [Talasila et al., 2024]. Its monorepo comprises a React web client, library, runner and logger microservices, and the `dtaas-services` service manager. Its defining idea is *composability*: a twin is assembled from reusable library assets — data, models, functions and tools — selected in the client and executed by the runner.

Three existing capabilities change what a visualisation layer must build rather than simply use. **A full data stack:** `dtaas-services` installs, in TLS mode, InfluxDB, Grafana, RabbitMQ as an AMQP broker *with its MQTT plugin enabled* — so one broker serves both protocols — MongoDB, ThingsBoard (IoT device management, with a PostgreSQL backend) and GitLab as OAuth2 identity provider and Git repository service. **Git in the browser:** the client already depends on `@gitbeaker/rest` 43.8, so it can read *and write* repository files under the same OAuth2 session, meaning versioned persistence needs no new backend. **Real Linux workspaces:** each user gets a container exposing Jupyter, VS Code, `ungit` and an xfce desktop over VNC *in the browser*, embedded through `react-iframe`; files a twin execution produces are addressable by URL.

Against that, the client's own dependencies are React 19.2, Vite 8, TypeScript 6, MUI 9, Redux Toolkit 2, React Router 7 and Monaco, with routes for `account`, `auth`, `config`, `digitaltwins`,

library, measurement and workbench — and *no* 3D, image-overlay or scientific-visualisation dependency at all.

Goal. A reusable React component package, `@dtaas/visualisation`, that (a) accepts visual substrates from arbitrary URLs — 3D assets, photographs, floor plans and live video — including assets served by the library microservice, GitLab or a workspace; (b) drives their visual state from heterogeneous live streams, reusing the brokers and databases already deployed; (c) serves several domains (city, machines, buildings, hospitals, and the marine and mobility domains evidenced in the literature reviewed) without a rebuild for each; and (d) couples *simulated* to *measured* state in a single view — the capability that distinguishes a digital twin from a monitoring dashboard.

Two requirements that are easy to under-weight. A large share of what users ask for is not a 3D model. They have a *photograph* — a traffic camera still, a room, a plant floor, a scanned plan — and want simulation and live readings drawn on it, correctly placed (technique **T13**). Where a camera exists they want the *live feed beside the simulation*, moving together in time, so divergence is directly observable (technique **T14**). Both are treated here as first-class substrates, which shapes the architecture in Section 3.

Out of scope. Dashboarding, which Grafana and ThingsBoard already do well — the right move is to *drive* them from the same clock, not rebuild them. And asset authoring: the package consumes substrates. The exceptions are binding authoring and image registration, because both bind a signal to a location and no external tool can do that for us.

Sources. This report consolidates a survey of commercial platforms (Autodesk Tandem and Platform Services, AWS IoT TwinMaker, Bentley iTwin.js, Samsara, NVIDIA Omniverse/OpenUSD); an audit of open-source packages with versions and licences verified against the npm registry and GitHub API on 12–13 September 2026; and eight peer-reviewed papers supplied in `sources/`. Appendix A records provenance, including three supplied files that are failed downloads rather than papers.

2 Problem

A web-based DT platform must render arbitrary visual substrates — 3D assets, photographs and live video — loaded from arbitrary URLs, driven by heterogeneous live data streams, across unrelated application domains, without rebuilding the stack each time a new domain or medium appears. No open-source package solves this: every candidate solves one domain deeply or all domains shallowly, and almost none treat an image or a video feed as a legitimate place to put simulation output.

2.1 Six difficulties

P1 Domain rebuild. 3D libraries are domain-shaped — a BIM viewer models storeys and IFC property sets, a geospatial engine models tiles and terrain, a robotics viewer models joints. Serving four domains means either one poor fit in three of them or four integrations to maintain. Rundel and De Amicis [2023] report the same pressure at city scale: operational city DTs with simulation (Virtual Singapore, Herrenberg) were *impracticable* for small municipalities — Virtual Singapore alone exceeded USD 73 million over five years — and their answer was procedural generation from existing public geodata rather than bespoke authoring. The lesson generalises: derive the scene from data that already exists.

P2 Anchoring. Something must connect a signal to a location. TwinMaker binds entity/component/property onto a GLB submodel; iTwin.js keys on Element ID; Tandem classifies

assets explicitly. All three *author* the link and persist it; runtime inference from coordinates or names breaks on re-export. DTaaS has an advantage here: ThingsBoard already holds a device and asset registry, so the anchor namespace need not be invented.

P3 Sustained update. Applying an encoding once is trivial; applying it to 10^4 – 10^6 elements at 10–60 Hz without rebuilding the scene graph is not. This constrains technique choice far more than aesthetics.

P4 Heterogeneity. Ali et al. [2024] name this as open research: combining 3D geometry, time series and map layers legibly, and visualising correlations across space and time, “needs more research”. Part of the problem is genuinely unsolved, not merely unbuilt.

P5 Simulation coupling. The problem commercial platforms serve worst. Tandem, Twin-Maker and Samsara visualise *measurement* at high polish; simulation lives in another tool with another viewer. Yet the residual between predicted and measured state is the high-value signal, because it represents a physics violation rather than a statistical oddity. Liao et al. [2025] co-visualise sensed and simulated deformation of a soft robot in one AR scene, reporting errors of 0–7.14%.

P6 Substrate plurality. Most libraries assume the answer is a 3D scene. In practice a large share of requests are a *photograph* (traffic still, room, plant floor, scanned plan) on which simulation should be drawn; a *live video feed* to be shown beside the simulation; or a *rendered output file* from a workspace execution that needs only placing and time-aligning. Treating these as second-class leaves the visualisation layer unusable until someone commissions a 3D model.

DTaaS is unusually placed on P5: its INTO-CPS lineage means FMI co-simulation already produces the predicted channel every surveyed commercial platform lacks, and InfluxDB already provides somewhere to put both channels. The cost of exploiting this is small provided measured and simulated state are two channels of one signal from the start.

2.2 Why the obvious approaches fail

One 3D library fails on P1 and P6. Pascal (@pascal-app/*, MIT) gives a full architectural editor but its document model is sites → buildings → levels → walls, in which a pump, a fjord or a photograph is not a citizen, and its viewer exposes no live-data binding. @thatopen/components has the best data semantics of any BIM option and is likewise BIM-shaped. CesiumJS is decisive at city scale and clumsy for one machine. **Everything on three.js** fails on effort, and is the wrong tool for annotating a JPEG. **A game engine** works — Rundel and De Amicis [2023] use Unity+SUMO via TraCI, Vasiljevic et al. [2024] Unity+Cesium — but produces an artefact hosted and versioned outside the platform; its proper home in DTaaS is the workspace desktop (§3.4). **Grafana and ThingsBoard alone** solve charting completely and the spatial problem not at all. **A commercial platform** does not solve P5, and only iTwin.js (MIT) is installable.

2.3 Requirements

ID	Requirement
R1	Substrate independence. Adding a domain or a medium means adding an adapter behind a stable interface, not forking the package.
R2	Authored anchoring. Signal-to-location bindings are explicit, persisted, versionable and human-editable.
R3	Transport agnosticism. MQTT and AMQP (both via the installed RabbitMQ), ThingsBoard telemetry, InfluxDB, SQL, HTTP and files normalise to one stream shape.
R4	Temporal addressing. Every encoding reads state at a playhead, never “latest value”.

ID	Requirement
R5	Dual-channel state. Measured and simulated values of one signal are first-class and separately addressable.
R6	Sustained update. $O(\text{changed elements})$ per frame, no scene-graph rebuild.
R7	Composability as a DTaaS asset. A visualisation is storable in the library, selectable in composition, shareable between users.
R8	Licence cleanliness. MIT, Apache-2.0 or BSD only; network copyleft is disqualifying for a platform client.
R9	Graceful degradation. A twin with no 3D asset still gets a usable view; a browser without WebGPU still renders.
R10	Reuse of the installed stack. Consume InfluxDB, RabbitMQ, ThingsBoard, Grafana and GitLab as deployed; no parallel broker, historian, dashboard or file store.
R11	Media synchronisation. Where video and simulation are shown together, their time alignment is explicit, measurable and correctable.

R8 eliminates `xeokit-sdk` and `realvirtual WEB` (both AGPL-3.0) and flags `Potree` and `loaders.gl` (NOASSERTION) for checking. **R10** is what most changes the plan relative to a greenfield design.

3 Solution Architecture

3.1 The invariant beneath every surveyed platform

Strip the branding from Tandem, TwinMaker, iTwin.js and Omniverse and the same three layers appear. **Anchor:** a stable identifier tying a signal to a location on a substrate — IFC GUID, glTF node, USD prim, Element ID, or a point or polygon in image coordinates. **Encode:** a rule turning a value into a visual property — colour, opacity, visibility, transform, flow speed, ghost offset, glyph, label; this is where commercial platforms put their product, and TwinMaker’s no-code rule editor *is* what customers pay for. **Sustain:** applying encodings continuously at scale without rebuilding the scene; iTwin.js devotes a native tile generator and a WebGL frontend to this layer alone.

The renderer is not in this list. It is an implementation detail of the sustain layer. Treating it as the centre of the architecture is what makes a visualisation package domain-locked — and what makes packages unable to treat a photograph or a video frame as a place where data can live.

3.2 Six layers

Layers 1–4 are domain- and substrate-independent and form the reusable core; layer 5 is a thin per-substrate adapter; layer 6 is third-party.

Layer 1 — Transport. Every source normalises to one event shape, with optional historical backfill:

```
export type Channel = 'measured' | 'simulated' | 'predicted' | 'setpoint';

export interface SignalSample {
  readonly twinId: string; readonly signalPath: string; // "ward3/ahu1/supplyTemp"
  readonly channel: Channel; readonly ts: number; // epoch ms, source-assigned
  readonly value: number | string | boolean | number[];
  readonly quality?: 'good' | 'stale' | 'bad';
}
```

```

}

export interface TransportAdapter {
  readonly id: string;
  connect(cfg: unknown): Promise<void>;
  subscribe(paths: string[], sink: (s: SignalSample) => void): () => void;
  range?(paths: string[], from: number, to: number): Promise<SignalSample[]>;
  disconnect(): Promise<void>;
}

```

Adapters follow the installed stack, not a wish-list: MQTT over WSS and AMQP over Web-STOMP, both against the single RabbitMQ; ThingsBoard telemetry over its WebSocket API; InfluxDB for `range()`; SQL/HTTP for the logger and arbitrary DT endpoints; and a file adapter replaying CSV or Parquet from the library, GitLab or a workspace URL — which makes the stack testable offline and lets an FMI output file serve as the `simulated` channel with no runtime integration.

Layer 2 — Temporal state store, hot and cold. A ring buffer per `(signalPath, channel)` holds the live window; when the playhead moves outside it, the store issues an InfluxDB `range()` query and serves from the returned window. Every consumer calls `valueAt(path, channel, t)`, never “current value”. This implements **R4** and unlocks 4D replay (**T6**), residuals (**T9**), image-overlay replay (**T13**) and video sync (**T14**) at low marginal cost. The playhead, subscriptions and connection status belong in Redux Toolkit; the sample buffers deliberately do not, because dispatching an action per sample destroys frame rate.

Having InfluxDB deployed converts the time store from a hard problem into a cache. Without a historian, honest replay means holding everything in memory or building a backend; with one, it is a windowed query behind an interface the rest of the package cannot distinguish from live data.

Layer 3 — Anchor resolution. Anchor kinds span three media, plus the imported registry:

```

export type AnchorKind =
  | 'ifc-guid' | 'gltf-node' | 'usd-prim' | 'instance-id' // 3D substrates
  | 'geo' // georeferenced
  | 'image-point' | 'image-region' | 'image-plane' // image
  substrates
  | 'video-panel' // video
  substrates
  | 'tb-entity'; // ThingsBoard
  device/asset

```

An anchor carries `signalPath`, `kind`, `ref`, a human `label` and, for image substrates, the `homographyId` of the calibration that applies. `tb-entity` matters: because ThingsBoard holds the registry, an anchor can pair *this device* with *that IFC element*, so the signal namespace is imported rather than invented and a rename does not silently break the binding.

Layer 4 — Encoding rules. Declarative, serialisable, and the genuinely reusable content of the package: `colorScale`, `visibility`, `transform`, `flow`, `residual`, `ghost`, `label` and `attention` for geometric substrates; `glyph`, `regionFill`, `fieldOverlay` and `trajectory` for image and video substrates. Because encodings are plain data, a complete visualisation is a JSON document — which is what makes **R7** achievable and, given `@gitbeaker/rest`, what makes it storable and reviewable with no new backend.

Layer 5 — Substrate adapters. The seam that delivers **R1**. Generalising from *renderer* to *substrate* is what admits images and video as peers of 3D:

```

export interface SubstrateAdapter {
  readonly id: string; readonly supports: AnchorKind[];
  mount(container: HTMLElement, descriptor: SubstrateDescriptor): Promise<void>
  >;
  resolve(anchor: Anchor): ElementRef | null;
  apply(ref: ElementRef, enc: ResolvedEncoding): void; // hot path, must be 0
  (1)
  frame(ref: ElementRef, opts?: FrameOptions): void;
  pick(x: number, y: number): ElementRef | null;
  setTime(t: number): void; // media seek,
  animation
  dispose(): void;
}

```

Adapter	Backing library	Substrate and domains
image	react-konva; OpenCV.js for calibration; Leaflet (CRS.Simple) for very large images	Photographs, floor plans, scanned drawings, rendered output frames. All domains; cheapest substrate, build first.
aec	@thatopen/components + @thatopen/fragments	IFC models: hospitals, buildings, plants.
mesh	react-three-fiber + drei (+ urdf-loader)	glTF assemblies: machines, robots, equipment.
video	HTMLVideoElement + hls.js or WebRTC	Live and recorded camera feeds beside or beneath simulation output.
geo	CesiumJS / 3DTilesRendererJS (+ deck.gl)	City, mobility, marine; anything georeferenced or tiled.
field	vtk.js	Simulation output: volumes, isosurfaces, streamlines. Own canvas, synchronised camera.
embed	react-iframe	Grafana panels, ThingsBoard dashboards and the workspace VNC desktop, driven by the shared playhead.

3.3 Mapping onto the deployed DTaaS stack

Service	Role	How the package uses it
RabbitMQ	Live transport, AMQP + MQTT from one broker	Two Layer-1 adapters against one deployment: mqtt.js over WSS, @stomp/stompjs over Web-STOMP. No second broker.
InfluxDB	Historian	Cold tier of the Layer-2 store via @influxdata/influxdb-client; makes T6 and T9 possible over arbitrary past windows.
ThingsBoard	Device/asset registry; telemetry	Anchor namespace (tb-entity); a second live transport; dashboards embeddable.

Service	Role	How the package uses it
Grafana	Charting	<i>Not</i> reimplemented. Embedded and driven by the shared playhead: the viewer writes the current window into the panel URL's from/to , so scrubbing the substrate moves the chart. T10 for near-zero cost.
GitLab	Versioned storage and identity	Visualisation assets, anchor maps and (via LFS) models and photographs read and written with @gitbeaker/rest under the existing OAuth2 session. A threshold change becomes a merge request.
MongoDB	Document store	Optional: large generated anchor maps, per-user view state that should not pollute Git history.
Library / runner	Files; DT execution	Resolves lib:// substrates (needs MIME types for .glb , .ifc , .frag); produces the simulated channel and rendered frames.

The highest-value integration in this table is the cheapest: driving an embedded Grafana panel from the visualisation playhead. It gives linked time-and-space navigation without writing a charting component, and respects the division of labour the marine DT pilot arrived at independently [Vasilijevic et al., 2024].

3.4 Persistence, the workspace, and composability

Because **@gitbeaker/rest** is already a client dependency and GitLab is already the identity provider, the visualisation asset needs no bespoke storage: the viewer reads **visualisation.json** and its substrate at a given ref, the binding editor commits changes back, review happens as a merge request diff, and large substrates live in Git LFS beside the configuration referencing them.

Some visualisation will never happen in a browser canvas — full CFD post-processing, ParaView, Blender, **rviz**, a Unity city build. All already run in the DTaaS workspace, whose xfce desktop is reachable over VNC inside the client. The architecture should embrace this on three conditions: the desktop mounts through the **embed** adapter like any other substrate; output files written to the workspace come back by URL and are consumed by the **image** or **mesh** adapter, so a heavyweight render becomes a navigable, time-aligned artefact for everyone else; and where a workspace tool cannot accept the client's playhead, the layout says the pane is independent rather than implying synchrony. This gives an honest answer to “can DTaaS do proper scientific visualisation?” — yes, in the workspace — without pretending the web layer will match ParaView.

Finally, DTaaS already treats data, models, functions and tools as composable library assets. The consistent move is to add a *visualisation* asset type, so a **visualisation.json** sits beside the models and functions of a twin and is picked in the same composition flow. An abbreviated example spanning three substrates:

```
{ "schemaVersion": "1.1", "name": "junction-7-live",
  "layout": { "type": "split", "panes": ["photo", "camera", "grafana"] },
  "substrates": {
    "photo": { "adapter": "image", "source": "git://twins/junction7/junction7
              .jpg",
              "calibration": { "type": "homography", "id": "h1",
                              "imagePoints":
                                [[412,880],[1502,872],[1710,1040],[210,1050]],
                              "worldPoints": [[0,0],[24,0],[24,12],[0,12]], "units": "m" }
              },
```

```

"camera": { "adapter": "video", "source": ".../junction7/index.m3u8",
            "latencyHintMs": 2500 },
"grafana": { "adapter": "embed", "source": "https://grafana.example/d/abc/
            junction7",
            "timeParams": [{"from", "to"} ] },
"transports": [
  { "adapter": "mqtt", "url": "wss://rabbitmq.example:15676/ws",
    "topics": ["dtaas/junction7/+/count"], "channel": "measured" },
  { "adapter": "influx", "org": "dtaas", "bucket": "junction7", "role": "
    history" },
  { "adapter": "file", "url": "lib://data/junction7-sumo-out.csv",
    "channel": "simulated" } ],
"anchors": [
  { "signalPath": "junction7/armA/queueLen", "kind": "image-region",
    "ref": "poly:armA", "homographyId": "h1", "label": "North arm queue" } ],
"encodings": [
  { "target": "junction7/armA/queueLen", "substrate": "photo",
    "encoding": { "type": "regionFill", "domain": [0,40], "scheme": "traffic"
    } },
  { "target": "junction7/armA/queueLen", "substrate": "photo",
    "encoding": { "type": "residual", "against": "simulated", "domain":
    [-8,8] } } ] }

```

A visualisation therefore becomes shareable (replace the photograph and the homography), reviewable (JSON in Git) and executable by the runner (a headless adapter emits the same overlay as an image file during a DT execution).

3.5 Design rules

1. The binding layer is the product; substrates are dependencies. Layers 1–4 must import no rendering, imaging or media library — enforce with a lint rule.
2. One clock. Scene, image overlay, video element and embedded Grafana panel all sample the playhead.
3. Encodings are data. Anything expressible only as code is a gap in the vocabulary.
4. Hot path outside Redux. Redux holds control state; per-frame sampling reads buffers directly.
5. Reuse before build. If InfluxDB, RabbitMQ, ThingsBoard, Grafana or GitLab does it, call it.
6. Every adapter lazily loaded. An image twin must never ship the BIM engine.

4 Visualisation Techniques by Domain

Fourteen techniques, ordered by depth of coupling between physical asset, simulation and live stream. **T1–T12** apply mainly to geometric substrates; **T13** and **T14** cover image and video.

T1 Semantic anchoring

Bind `signalPath` → element using identifiers already in the model. Everything downstream is a lookup on this map; the inverse lookup (picking) makes the scene interrogable. *All domains — precondition, not option.*

T2 Feature overrides

Recolour, fade or hide individual elements via a per-element attribute read by one shared shader, keeping cost proportional to *changed* elements. iTwin.js `FeatureOverrideProvider`; TwinMaker model shader. **Highest-leverage technique: T3, T9 and T12 all consume it.**

T3 Sparse-sensor field mapping

Twenty sensors, one continuous surface. Interpolate (IDW, RBF, nearest) into a texture sampled by the surface shader. The interpolation is the honest part and must expose its uncertainty. *Hospitals, buildings, marine.*

T4 Volumetric simulation fields

CFD, thermal, acoustic or dispersion results as isosurfaces, cutting planes or ray-marched volumes registered to the sensor geometry. Absent from Tandem, TwinMaker and Samsara; central to [Vasilijevic et al. \[2024\]](#), who render OpenDrift and DREAM forecasts into the twin. *Marine, city, buildings, machines.*

T5 Flow advection

Rate and direction as movement — particles in a pipe, arrows in a duct, vehicles on a link. Distinguishes “flowing slowly” from “stopped”, which a colour ramp cannot. TwinMaker ships it as the *motion indicator*. *Buildings, city, marine, machines.*

T6 4D replay

One clock drives geometry, appearance, camera and media; scrubbing moves the whole scene. iTwin.js `RenderSchedule`; Samsara fuses GPS, multi-camera video and sensors on one timeline where detected *events*, not timestamps, are the navigation anchors. *All; critical for machines and mobility.*

T7 Reality-capture fusion

Register as-built capture against as-designed model — point clouds where geometry must be measurable, Gaussian splats where fidelity is the goal. [Mobility DT Visualization](#) reconstruct vehicle geometry from crowdsourced imagery via NeRF precisely because authoring is “labour-intensive and time-consuming”, and report PSNR falling 53–54% outside ideal capture — a caution against assuming capture quality. *Buildings, city, mobility.*

T8 Video projection

Project a live feed onto the surfaces it sees, or lift detections out of video into the scene as entities. Samsara Site Visibility is this productised. *Hospitals (privacy-constrained), machines, city.*

T9 Ghost twin / residual

Render simulated and measured state in one frame — a translucent predicted pose against the measured one, or surfaces coloured by residual rather than value. Essentially unserved commercially; Omniverse comes closest via USD layers. Core to [Liao et al. \[2025\]](#). **Machines above all, and where DTaaS has a genuine advantage.**

T10 Linked abstraction levels

Schematic, plan, table and 3D as projections of one selection and one clock. [Ali et al. \[2024\]](#) identify the general case as open research. *DTaaS shortcut: drive an embedded Grafana panel’s from/to from the playhead.*

T11 Streaming LOD

Hierarchical tiles, compression, per-element batching — the precondition for everything else at real scale. *City and buildings critically.*

T12 Attention routing

The scene navigates itself to the anomaly: camera frames the element, the rest drops to x-ray. Almost entirely rules plus camera interpolation. *Hospitals and machines especially.*

T13 Image-substrate overlay

A photograph replaces the model; readings, simulated flows, queue lengths and predictions are drawn *on the image* and animate with the playhead. Three tiers, and choosing the

cheapest sufficient one is most of the skill: (i) **pixel anchoring** — points and polygons in image coordinates, adequate when the question is “what is the value *here*”; (ii) **ground-plane homography** — four image/world correspondences on a common plane yield a 3×3 homography, which is what makes simulation output directly projectable (SUMO positions, crowd flow, predicted queue extent) in metres rather than pixels; (iii) **full camera pose** — intrinsics plus extrinsics for true 3D overlay, worth it only when objects leave the ground plane. *All domains; disproportionately valuable for city and mobility.*

T14 Live video beside simulation

Three arrangements: side-by-side on a shared clock (video seeked to t minus a declared offset); overlaid, which is **T13** with a moving image; or a swipe divider between camera and matched-viewpoint render. The hard part is *time*, not display: stream latency runs from hundreds of milliseconds (WebRTC) to seconds (HLS), so **R11** demands the offset be declared, measurable and adjustable. A side-by-side view with unstated skew is worse than no view, because latency-induced divergence is indistinguishable from a wrong model. *City, mobility, machines, marine.*

T13 should be built before any 3D substrate. It exercises the whole binding layer at a fraction of the cost, satisfies **R9** by giving every twin some usable view, and is what many users were asking for in the first place.

Table 4: Technique applicability by domain.

ID	Technique	Hospitals	Buildings	Machines	City/mob.	Marine
T1	Semantic anchoring	Critical	Critical	Critical	Critical	Critical
T2	Feature overrides	Critical	Critical	High	High	Medium
T3	Surface field mapping	Critical	High	Low	Medium	High
T4	Volumetric sim. fields	Medium	High	High	High	Critical
T5	Flow advection	Medium	High	High	Critical	High
T6	4D replay	High	High	Critical	Critical	High
T7	Reality-capture fusion	Medium	High	Medium	High	Medium
T8	Video projection	High	Low	High	High	Medium
T9	Ghost twin / residual	Medium	Medium	Critical	Medium	High
T10	Linked abstraction levels	High	Medium	High	High	High
T11	Streaming LOD	High	Critical	Low	Critical	Medium
T12	Attention routing	Critical	High	High	High	Medium
T13	Image-substrate overlay	High	High	High	Critical	Medium
T14	Live video + simulation	High	Low	High	Critical	High

Three readings matter. **Hospitals and buildings are one rendering problem:** both IFC-shaped, served by one adapter; what differs is data semantics, which lives in the anchor and encoding layers. **Machines are genuinely different** — no BIM library helps with a pump or a robot cell — but the scope is smaller because there is no semantic model to honour, and **T9** pays off most there. **City, mobility and marine share a renderer**, all georeferenced and leaning on **T4** and **T5**; both [Rundel and De Amicis \[2023\]](#) and [Vasiljevic et al. \[2024\]](#) independently arrive at the same stack shape. And **T13** scores high in every column, because a photograph exists long before a model does.

5 Open-Source Packages

Versions and licences verified against the npm registry and the GitHub API on 12–13 September 2026.

Table 5: Recommended package per technique (abridged).

ID	Package	Version	Licence	Role
T1	@thatopen/components	3.4.8	MIT	IFC property sets, classifier, lookup
T1	three-mesh-bvh	0.9.x	MIT	Fast picking; the inverse lookup
T2	three.js InstancedMesh	—	MIT	Per-instance colour, one draw call
T2	@thatopen/components-front	3.4.4	MIT	Highlighter over fragments
T3/T4	vtk.js	32.x	BSD-3	Interpolation, volumes, contours, streamlines
T3	deck.gl	9.4.0	MIT	HeatmapLayer, plan-view equivalent
T5	deck.gl TripsLayer	9.4.0	MIT	Time-windowed paths
T6	@reduxjs/toolkit	2.x	MIT	Playhead and control state (already present)
T7	Spark	current	MIT	Gaussian-splat renderer for three.js
T7/T11	3DTilesRendererJS	0.4.x	Apache-2.0	Streamed tiles and point clouds
T9	—	—	—	No package exists ; built on T2 + T6
T11	@thatopen/fragments	3.4.7	MIT	Batched IFC, per-element addressing
T12	@react-three/drei	10.x	MIT	CameraControls.fitToBox
T13	react-konva / konva	19.2.7 / 10.5.0	MIT	Glyph, polygon and label layer over a photograph
T13	opencv-ts (OpenCV.js)	1.3.6	MIT / Apache-2.0	findHomography, reprojection error
T13	leaflet + react-leaflet	1.9.4 / 5.0.0	BSD-2 / MIT	CRS.Simple for very large plans
T14	hls.js	1.7.3	Apache-2.0	HLS where latency tolerance is seconds
T14	MediaMTX	—	MIT	<i>Server-side:</i> RTSP → WebRTC/HLS gateway

Renderer verdicts. Adopt react-three-fiber+drei (MIT) as mesh; @thatopen/components (MIT) as aec — the only option whose core abstraction is “elements with queryable properties”, which is exactly a telemetry binding, though it is on v3 and churns, so pin versions; CesiumJS (Apache-2.0) as geo; vtk.js (BSD-3) as field, in its own canvas with a synchronised camera. **Study but do not adopt iTwin.js** (MIT): the entire technique catalogue already built, but a heavy platform-shaped dependency — its display-style / feature-override split is the cleanest model of the encode layer in existence. **Reconsider later @pascal-app/*** (MIT) if in-client twin *authoring* is ever wanted. **Fallback @speckle/viewer** (Apache-2.0) if building geometry arrives

from many CAD tools rather than as IFC. **Reject** `Babylon.js` (costs the R3F ecosystem, buys nothing), `GoJS` (2D and commercial); `foxxglove/studio` is archived and closed.

Transport and platform packages. `mqtt` 5.x (MIT), `@stomp/stompjs` 7.x (Apache-2.0), `@influxdata/influxdb-client` 1.35.0 (MIT), `@gitbeaker/rest` 43.8 (MIT, *already shipped*), `react-iframe` 1.8.5 (MIT, *already shipped*), `comlink` 4.4.2 (Apache-2.0), and the ThingsBoard WebSocket API, which needs no client library. Two of seven are already dependencies: the data plane is mostly configuration, not new dependency surface.

Gaps and licence notes. **T9** has no package and must be written — about a day of shader and encoder work, but only *after* **T2** and **T6** exist. **T8** has no maintained package (`three-projected-material` has not been pushed since July 2024; write the ≈ 80 -line projective shader instead). **T13** has no integrated package: `Konva` draws, `OpenCV` solves, `Leaflet` pans, but nothing binds a live signal to an image region along a playhead. And nothing at all covers Layers 1–4, which is why they are the package’s reason to exist. On licences: `xeokit-sdk` and `realvirtual WEB` are AGPL-3.0 and excluded — though `realvirtual`’s GLB-plus-signal wire format is worth studying; `Potree` and `loaders.gl` report `NOASSERTION` and need checking; `konva` and `hls.js` report `NOASSERTION` on GitHub while declaring MIT and Apache-2.0 on npm, so record the exception explicitly rather than letting a scanner fail the build.

6 Implementation Plan

The deliverable is a standalone npm package, `@into-cps-association/dtaas-visualisation`, published beside the other DTaaS packages. Standalone publication is deliberate: it forces the binding layer to be independent of the client’s routing, authentication and MUI theme, which is what makes it reusable by other web-based DT platforms and testable without spinning up DTaaS. Effort assumes one developer at ≈ 0.6 FTE; figures are planning estimates.

Table 6: Phases.

Phase	Deliverable	Techniques	Exit criterion
0 3 weeks	Layers 1–4 against the installed stack; <i>no substrate at all</i>	—	Fixture replay emits the correct resolved encodings over a scrubbed range, with no canvas on screen. If that is testable, R1 holds.
1 3 weeks	image adapter, calibration UI, Grafana embed	T13, T10	Live MQTT values drawn on a traffic or room photograph; scrub moves the embedded Grafana panel. No 3D in the build.
2 4 weeks	aec adapter; ThingsBoard registry import	T1, T2, T11	A real building or hospital-floor twin recolouring by measured value through the same binding layer.
3 2 weeks	Library asset type; binding editor committing via gitbeaker	T12	A second user reuses the asset by swapping substrate and anchors; a threshold change lands as a merge request.
4 4 weeks	Replay over InfluxDB; ghost twin; mesh adapter	T6, T9	Scrubbing moves geometry, image overlay, table and residual together; predicted and measured comparable in one frame.

Phase	Deliverable	Techniques	Exit criterion
5 3 weeks	video adapter; latency-offset control	T14, T8	Camera and simulation on one clock with skew displayed, adjustable and persisted.
6 4–6 weeks	geo and field adapters (demand-driven)	T3, T4, T5, T7	A georeferenced twin and a volumetric result render through the Phase-0 binding layer.

*Phase 0 is the phase most likely to be cut under schedule pressure and the one that must not be. Every later phase is cheap because of it, and neither the **channel** field nor the playhead can be retrofitted without touching every module written after them.*

Integration points in the client. A `visualise` sub-route under `client/src/route/digitaltwins/`; `visualisation` registered as an asset type in `route/library/` and the composition cart; a “show on substrate” affordance and shared playhead in `route/measurement/`; sibling VNC and web panes in `route/workbench/`; a `visualisationSlice` in `client/src/store/` holding playhead, connections, subscriptions, selection and layout — with sample buffers outside Redux; extended gitbeaker access in `client/src/util/`; and MIME types plus range requests in `servers/lib`.

Principal risks. Two are specific to the new substrates and both are failure modes of *credibility* rather than of code. **Homography error mistaken for model error:** show reprojection error at calibration time, store it with the homography, and refuse world-coordinate overlays above a threshold. **Video latency mistaken for divergence** (high likelihood, high impact): declare the offset per feed, display it, make it adjustable, and mark the pane “unsynchronised” when unknown. The others are ordinary: `@thatopen/*` API churn (pin versions; confine contact to the `aec` adapter); frame-rate collapse from per-sample Redux dispatch (buffers outside Redux from Phase 0; CI performance test at 60 Hz and 10^4 elements); bundle size across seven adapters (lazy-load, CI size budget); InfluxDB query cost on wide scrubs (window-and-cache, downsample, never query per frame); anchor drift on re-export (persist `kind` and `label`, surface a repair UI on failure); licence contamination (CI allowlist); and privacy in hospital and public-space video (restricted channels, default off, documented lawful basis — Ali et al. [2024] note location data alone can identify an individual).

What to build first. If one sentence survives contact with the schedule: **build the transport, the hot/cold time store and the SubstrateAdapter interface against the RabbitMQ and InfluxDB already deployed, then prove them on a photograph** — not on a 3D model. That route reaches a user-recognisable result in about six weeks, needs no WebGL, exercises every layer the 3D substrates will later reuse, and leaves a fallback view for every twin that never acquires a model.

7 Evaluation

Mobility DT Visualization separate *fidelity* (precision independent of latency) from *physical-to-digital synchronicity* (discrepancy caused by latency); the separation is the useful move, because the two trade against each other and one “quality” number hides the trade. Their own measurement makes the point: compressing offloaded imagery from > 18 MB to ≈ 6 MB cost less than 0.1% of PSNR while cutting latency substantially. The analogues here are *encoding fidelity* (rendered property versus the encoding evaluated in the pure layer — an automatable readback test) and *synchronicity* (p50/p95/p99 publish-to-pixel latency at a stated element count).

Three further measures follow from the new substrates and from **T9**. **Residual credibility**: report the residual's own error budget beside it, since a ghost twin showing 2 °C divergence is useless if pipeline uncertainty is 3 °C — [Liao et al. \[2025\]](#) model this discipline with quantitative metrics rather than visual plausibility. **Registration accuracy**: homography reprojection error, expressed in ground-plane metres as well as pixels, because three pixels near the horizon of a traffic photograph can be several metres. **Synchronisation skew**: the signed video-frame-to-playhead difference, reported continuously since transport latency drifts; the question is not whether skew is zero but whether it is known, bounded and smaller than the phenomenon being compared. Finally, [Chen et al. \[2024\]](#) evaluate an MR airport digital tower with ten licensed controllers on workload, situational awareness and trust — the right constructs for an operator-facing view, and not substitutable by usability, since a view can be easy to use and still degrade situational awareness by drawing attention to the wrong thing. All but the operator study belong in CI from the phase that introduces them.

8 Conclusion

8.1 Findings

1. The DTaaS client has no spatial visualisation capability today beyond embedded time series.
2. Every surveyed commercial platform shares one architecture — anchor, encode, sustain — and sells a no-code editor over the encode layer.
3. The renderer is therefore the wrong thing to standardise on. Transport normalisation, the temporal store, anchor resolution and the encoding vocabulary are the durable artefacts, and none exist off the shelf.
4. Not every substrate is a 3D scene. Photographs and live video are first-class places to put simulation output; generalising the adapter interface costs nothing and changes what ships in the first two months.
5. Roughly a third of the layer is already deployed — historian, broker, device registry, charting, versioned storage and identity need to be *used*, not built.
6. Hospitals and buildings are one 3D problem, machines another, city/mobility/marine a third: three geometric adapters plus `image`, `video` and `embed`.
7. Simulation–measurement co-visualisation is open ground, and DTaaS is unusually placed to take it.
8. One early decision governs half the catalogue: every encoding must read `valueAt(playhead)`, never “latest value”.
9. Licence discipline removes two of the strongest candidates (`xeokit-sdk`, `realvirtual WEB`) and flags two more.

8.2 Positioning and next step

The literature converges on this architecture from several directions. [Rundel and De Amicis \[2023\]](#) show that the escape from domain-rebuild cost is to derive the scene from data that already exists — and a photograph is data that already exists. [Vasilijevic et al. \[2024\]](#) adopt a deliberately two-tier visualisation, Grafana for series and a game-engine/Cesium layer for geospatial rendering, which is almost exactly the split proposed here between the `embed` adapter and the geometric substrates, coupled through a shared clock rather than merged. [Ali et al. \[2024\]](#) caution that **T10** is not merely engineering. [Mobility DT Visualization](#) supply the evaluation constructs and, by measuring how expensive geometry reconstruction is, the strongest argument for using imagery

where it suffices. [Liao et al. \[2025\]](#) demonstrate the ghost twin in miniature; [Chen et al. \[2024\]](#) supply the human-factors constructs and the case for shared, multi-operator views; and [Yang et al. \[2024\]](#) identify data integration and quality, not rendering, as the binding constraint on DT adoption — consistent with the claim that the binding layer is the product.

Next step. Build Phase 0 against the services already running, then Phase 1 on a photograph. Six weeks to something a user recognises as their own twin, no WebGL required, and the whole binding layer proven before a single 3D asset is commissioned.

A Source Provenance and Licence Register

A.1 Supplied sources

Eight of the eleven files in `sources/` are papers and were text-extracted successfully: [Yang et al. \[2024\]](#) (*Digital Twins in Construction*, Buildings 14(9):2616); [Ali et al. \[2024\]](#) (*Spatial Digital Twins*, PFG 92:761–778); [Vasilijevic et al. \[2024\]](#) (*A Digital Twin of the Trondheim Fjord*, JMSE 12(9):1530); [Chen et al. \[2024\]](#) (*Tangible digital twin with shared visualization for collaborative air traffic management operations*, Transportation Research Part C); [Liao et al. \[2025\]](#) (*An augmented reality-enabled digital twin system for reconfigurable soft robots*, Computers in Industry); [Rundel and De Amicis \[2023\]](#) (*Leveraging digital twin and game-engine for traffic simulations and visualizations*, Frontiers in Virtual Reality 4:1048753); and [Mobility DT Visualization](#) (*Visualization of Mobility Digital Twin*, IEEE).

Three caveats are recorded rather than hidden. `1-s2.0-S0278612525000834-main.pdf` uses CID-encoded embedded fonts and could not be text-extracted by either the direct extractor or Word conversion; it is identified from metadata as [IIoT factory-machine DT](#) and *nothing in this report relies on its content*. And `dam-visualisation.pdf`, `main.pdf` and `main-2.pdf` (each ≈ 61 KB) are **not papers** — each is a saved ScienceDirect “JavaScript is disabled / Request Verification” error page.

*If `dam-visualisation.pdf` was meant to contribute a dam or hydraulic-structure case study — a domain not otherwise covered — it should be re-downloaded and this report extended, since that domain would exercise **T4** and **T5** heavily.*

A.2 Extraction and verification method

No LaTeX distribution, Python, Node.js or Poppler was available when the sources were read. PDF text was recovered with a purpose-written PowerShell routine that walks each file’s stream objects, inflates `FlateDecode` payloads via `System.IO.Compression.DeflateStream`, and extracts text-showing operators (`Tj`, `TJ`) with PDF string-escape decoding. Inter-token spacing in the recovered text is imperfect but word content is faithful; quotations were re-spaced by hand and checked against context. Package versions and licences were read from the npm registry and the GitHub REST API on 12–13 September 2026; where the API reports `NOASSERTION` — no determinable SPDX identifier — that is stated rather than guessed.

A.3 Licence register

Package	Licence	Status for DTaaS
<code>react-three-fiber</code> , <code>dreif</code> , <code>three.js</code> , <code>three-mesh-bvh</code>	MIT	Clear
<code>@thatopen/components</code> , <code>-front</code> , <code>fragments</code>	MIT	Clear
<code>deck.gl</code> , <code>Spark</code> , <code>SuperSplat</code>	MIT	Clear

Package	Licence	Status for DTaaS
iTwin.js, @pascal-app/*	MIT	Clear; not adopted (see §5)
react-konva, react-leaflet,	MIT	Clear
opencv-ts		
mqtt,	MIT	Clear
@influxdata/influxdb-client		
@gitbeaker/rest,	MIT	Clear; already client dependencies
react-iframe		
MediaMTX	MIT	Clear; deployment component, not a client dependency
CesiumJS, 3DTilesRendererJS	Apache-2.0	Clear
@speckle/viewer,	Apache-2.0	Clear
@stomp/stompjs, comlink		
OpenCV (behind opencv-ts)	Apache-2.0	Clear
vtk.js	BSD-3-Clause	Clear
leaflet	BSD-2-Clause	Clear
konva	MIT (npm); NOASSERTION (GitHub)	Clear; record as a CI exception
hls.js	Apache-2.0 (npm); NOASSERTION (GitHub)	Clear; record as a CI exception
Potree, loaders.gl	NOASSERTION	Check before use
xeokit-sdk, realvirtual WEB	AGPL-3.0	Excluded — network copyleft
GoJS	Commercial	Excluded

References

- Mohammed Eunus Ali, Muhammad Aamir Cheema, Tanzima Hashem, Anwaar Ulhaq, and Muhammad Ali Babar. Spatial digital twins: Technologies, applications and research directions. *PPG – Journal of Photogrammetry, Remote Sensing and Geoinformation Science*, 92:761–778, 2024. doi: 10.1007/s41064-024-00301-2.
- K. Chen et al. Tangible digital twin with shared visualization for collaborative air traffic management operations. *Transportation Research Part C: Emerging Technologies*, 2024. ScienceDirect PII S0968090X24000676.
- IIoT factory-machine DT. Iiot-enabled digital twin for legacy and smart factory machines with llm integration. *Journal of Manufacturing Systems*, 80, 2025. ScienceDirect PII S0278612525000834. Cited from metadata only; the supplied PDF could not be text-extracted.
- Zhongyuan Liao, Wanzhen Wei, Lei Han Zhang, Yuer Gao, and Yi Cai. An augmented reality-enabled digital twin system for reconfigurable soft robots: Visualization, simulation and interaction. *Computers in Industry*, 2025. ScienceDirect PII S0166361525000508.
- Mobility DT Visualization. Visualization of mobility digital twin: Framework design, case study, and future challenges. IEEE; retrieved via IEEE Xplore, 2023. Edge-assisted NeRF pipeline; defines fidelity and physical-to-digital synchronicity.
- Samuel Rundel and Raffaele De Amicis. Leveraging digital twin and game-engine for traffic simulations and visualizations. *Frontiers in Virtual Reality*, 4:1048753, 2023. doi: 10.3389/frvir.2023.1048753.

- Prasad Talasila, Cláudio Gomes, Lars B Vosteen, Hannes Iven, Martin Leucker, Santiago Gil, Peter H Mikkelsen, Eduard Kamburjan, and Peter G Larsen. Composable digital twins on digital twin as a service platform. *SIMULATION*, page 00375497241298653, 2024. doi: 10.1177/00375497241298653.
- Antonio Vasilijevic, Ute Brønner, Muriel Dunn, Gonzalo García-Valle, Jacopo Fabrini, Ralph Stevenson-Jones, Bente Lilja Bye, Igor Mayer, Arne Berre, Martin Ludvigsen, and Raymond Nepstad. A digital twin of the trondheim fjord for environmental monitoring—a pilot case. *Journal of Marine Science and Engineering*, 12(9):1530, 2024. doi: 10.3390/jmse12091530.
- Zhou Yang, Chao Tang, Tongrui Zhang, Zhongjian Zhang, and Dat Tien Doan. Digital twins in construction: Architecture, applications, trends and challenges. *Buildings*, 14(9):2616, 2024. doi: 10.3390/buildings14092616.